

CyberAudit QA — Escopo do projeto de testes

Sep 29, 2026 · @João

Objetivo

Entregar um repositório público `cyberaudit-qa` que prove, com código executável e CI verde, domínio de Postman/Newman, RestAssured, Cypress e Selenium sobre um sistema real em produção.

O que o recrutador precisa encontrar no repositório:

- Quatro suítes rodando no GitHub Actions a cada push, com relatório Allure publicado.
- README com a estratégia de teste, cobertura por área e como rodar tudo localmente com um comando.
- Artefatos de processo: plano de teste, casos de teste e bug reports com template.
- Bugs reais encontrados no próprio CyberAudit, registrados como issues e corrigidos.

Prazo estimado: 5 a 8 semanas. Confiança baixa, pois depende das horas semanais disponíveis.

Regras obrigatórias

Cinco regras valem para todas as fases; quebrar qualquer uma invalida o projeto como evidência.

1. **Nunca rodar testes contra produção** (`cyberauditapp.com`). Rate limit, 2FA e scans ativos são reais lá, e clientes seriam afetados.
2. **Varredura ativa só contra alvos locais próprios**, subidos no ambiente de teste.
3. **Autoria real**. Cada teste é escrito ou reescrito por você e explicável linha a linha. Commits no seu usuário do GitHub.
4. **Sem segredos no repositório**. Credenciais e seeds TOTP apenas de teste, via variáveis de ambiente e GitHub Secrets.
5. **CV acompanha o código**. Uma ferramenta só sai de "em estudo" quando a fase dela cumprir o critério de pronto.

Ambiente de teste e estrutura

Tudo roda num `docker-compose.test.yml` com cinco serviços: backend (profile `test`), frontend, PostgreSQL, OWASP Juice Shop como alvo vulnerável e WireMock como alvo controlado.

O profile `test` do backend precisa de:

- Um usuário de seed por plano (Free, Pro e acima), com senha e seed TOTP fixos e conhecidos.
- Domínios dos alvos locais já marcados como verificados.
- Reset do banco entre execuções (script de truncate ou migração limpa).
- Limites de rate lidos da configuração, para os testes validarem contra o valor configurado em vez de um número fixo.

Ponto crítico: a guarda anti-SSRF vai bloquear os alvos locais, porque estão na rede privada do Docker. A solução é uma allowlist configurável, ativa só no profile `test`. Crie também um teste provando que essa allowlist não existe fora do profile `test`; caso contrário, você abre uma brecha de SSRF em produção.

Estrutura do repositório:

```
cyberaudit-qa/
├── docs/           plano de teste, casos de teste, template de bug
├── env/           docker-compose.test.yml, seed.sql, mappings do WireMock
├── api-postman/    collection.json, environment.local.json
├── api-restassured/ Maven, JUnit 5, RestAssured, AssertJ, Allure
├── e2e-cypress/    TypeScript, otplib para gerar o código TOTP
├── e2e-selenium/   Maven, Selenium WebDriver, Page Object Model
└── .github/workflows/
```

Se o backend do CyberAudit ainda não tem testes unitários e de integração (JUnit, Mockito, Testcontainers), eles pertencem ao repositório principal, não a este.

Fases

Sete fases em sequência, ordenadas por retorno por hora investida: API antes de interface, Java antes de JavaScript.

Fase	Ferramenta	Entregáveis	Pronto quando	Estimativa
0. Planejamento	Markdown	Plano de teste, matriz requisito → caso, template de bug	30+ casos catalogados com prioridade	2–3 dias
1. Ambiente	Docker Compose	Compose de teste, seed, alvos locais, allowlist SSRF só no profile <code>test</code>	Um comando sobe tudo e o health check passa	3–5 dias

Fase	Ferramenta	Entregáveis	Pronto quando	Estimativa
2. API (collection)	Postman + Newman	Collection por área, environments, testes de status e payload	Newman roda no CI e quebra o build quando um teste falha	2-4 dias
3. API (código)	RestAssured + JUnit 5	Suítes de auth, controle de acesso, planos, rate limit, SSRF, scan e relatório; validação por JSON Schema	Todos os casos P0/P1 de API automatizados, CI verde	1-2 semanas
4. E2E	Cypress	Fluxos de login + 2FA, scan, histórico, agendamento e PDF; seletores <code>data-testid</code>	6+ fluxos estáveis, zero falhas intermitentes em 10 execuções seguidas	1-2 semanas
5. E2E Java	Selenium + POM	3-4 fluxos críticos, headless	Roda no CI em modo headless	~1 semana
6. CI e relatório	GitHub Actions + Allure	Pipeline completo, relatório no GitHub Pages, badge no README	Link público do relatório funcionando	2-3 dias

Selenium fica por último porque duplica a cobertura do Cypress. Só vale se as vagas-alvo pedirem Selenium.

Catálogo de casos prioritários

São 26 casos iniciais: 15 P0, 9 P1 e 2 P2. P0 cobre segurança e regras de negócio centrais; se um P0 falhar, é bug a registrar, não teste a ajustar.

Os códigos HTTP citados são o esperado pelo padrão de mercado. Confirme o que a API retorna hoje antes de fixar nos testes.

ID	Área	Cenário	Resultado esperado	Ferramenta	Prioridade	Status
AUTH-01	Autenticação	Login com credenciais válidas	JWT emitido, acesso liberado	Postman, RestAssured	P0	▼

ID	Área	Cenário	Resultado esperado	Ferramenta	Prioridade	Status
AUTH-02	Autenticação	Senha errada e usuário inexistente	401 com a mesma mensagem nos dois casos, sem revelar se o usuário existe	RestAssured	P0	▼
AUTH-03	Autenticação	JWT expirado, assinatura adulterada, alg: none	401 em todos	RestAssured	P0	▼
AUTH-04	2FA	Sem código TOTP, código inválido, código reutilizado	Acesso negado nos três	RestAssured, Cypress	P0	▼
AUTH-05	API key	Chamada com API key revogada	401	Postman, RestAssured	P1	▼
AUTHZ-01	Controle de acesso	Usuário A consulta scan do usuário B pelo ID	403 ou 404, nunca os dados	RestAssured	P0	▼
PLAN-01	Planos	Usuário Free solicita relatório PDF	Bloqueado (recurso Pro)	RestAssured, Cypress	P0	▼
PLAN-02	Planos	Exceder a cota de scans do plano	Bloqueado com mensagem clara	RestAssured	P1	▼
RATE-01	Rate limit	Exceder o limite configurado por usuário/IP	429; volta a 200 após a janela	RestAssured	P1	▼
SSRF-01	Anti-SSRF	Alvo 127.0.0.1, localhost, ::1	Rejeitado	RestAssured	P0	▼
SSRF-02	Anti-SSRF	Alvo 169.254.169.254, 10.x, 192.168.x	Rejeitado	RestAssured	P0	▼
SSRF-03	Anti-SSRF	Formas alternativas: 2130706433, 0x7f000001, 127.1	Rejeitado	RestAssured	P0	▼
SSRF-04	Anti-SSRF	Hostname público que resolve para IP privado	Rejeitado	RestAssured	P0	▼
SSRF-05	Anti-SSRF	Alvo externo que redireciona para IP interno	Redirecionamento não seguido	RestAssured + WireMock	P1	▼
SSRF-06	Anti-SSRF	Allowlist de teste fora do profile test	Allowlist inexistente	RestAssured	P0	▼
DOM-01	Domínio	Varredura ativa em domínio não verificado	Recusada	Postman, RestAssured	P0	▼

ID	Área	Cenário	Resultado esperado	Ferramenta	Prioridade	Status
SCAN-01	Motor	Varredura passiva no alvo controlado	Módulos esperados com status OK	RestAssured	P0	▼
SCAN-02	Motor	Alvo com atraso forçado no WireMock	Módulo marcado como timeout, relatório indica resultado parcial	RestAssured + WireMock	P1	▼
SCAN-03	Motor	Varredura ativa no Juice Shop	XSS e SQL injection detectados	RestAssured	P0	▼
SCAN-04	Motor	Alvo limpo com headers corretos	Nenhum falso positivo nesses módulos	RestAssured + WireMock	P1	▼
SCORE-01	Pontuação	Scan com achado crítico e o resto limpo	Nota não fica alta (override de severidade)	RestAssured	P0	▼
REP-01	Relatório	Download do PDF por usuário Pro	application/pdf válido, contém os achados do scan (checar com PDFBox)	RestAssured	P1	▼
REP-02	Relatório	Achado conhecido no PDF	Mapeamento LGPD/ISO 27001 presente	RestAssured	P2	▼
UI-01	Interface	Login completo com 2FA	Chega ao dashboard	Cypress, Selenium	P1	▼
UI-02	Interface	Iniciar scan, acompanhar e ver no histórico	Resultado e nota exibidos	Cypress, Selenium	P1	▼
UI-03	Interface	Criar, editar e remover agendamento recorrente	Alterações persistem após recarregar	Cypress	P2	▼

CI, relatórios e marcos do CV

O pipeline roda a cada push: sobe o ambiente, executa as quatro suítes em paralelo e publica um relatório Allure unificado no GitHub Pages.

- Em falha, guardar screenshots e vídeos do Cypress como artefatos do job.
- Teste intermitente vai para quarentena com uma issue aberta; nunca é apagado.
- Bug encontrado no CyberAudit vira issue com o template, depois correção e teste de regressão. Isso rende a melhor história de entrevista do projeto.

Cada marco libera uma mudança específica no CV de QA:

Marco	Mudança no CV
Fase 2 pronta	Postman sai de "em estudo": testes de API com Postman/Newman integrados ao CI
Fase 3 pronta	Novo bullet no CyberAudit: suíte RestAssured/JUnit 5 cobrindo autenticação, controle de acesso, rate limiting e anti-SSRF, no GitHub Actions
Fase 4 pronta	Cypress sai de "em estudo": testes E2E dos fluxos críticos, incluindo 2FA
Fase 5 pronta	Selenium sai de "em estudo", com Page Object Model
Fase 6 pronta	Link do repositório e do relatório Allure no CV e no LinkedIn

Os bullets finais levam números reais: quantidade de casos automatizados e de bugs encontrados e corrigidos.